

Subiecte 2 1

Discutat in discutie 2

Subiecte 2 2

Discutat in discutie 2

Subiecte 2 3

Discutat in discutie 2

Subiect 2 4

1

- Funcțional: Sistemul detectează ploaia și activează automat ștergătoarele.
- Non-funcțional: Reacțiile trebuie generate în timp real (sub 100 ms latency).

Arhitectură: Event-Driven + Microservices

- Stil arhitectural: Event-Driven Architecture (EDA) pentru răspunsuri rapide și decuplare.
- Justificare:
 - Senzorii (input-uri) trimit evenimente.
 - Servicii reactive prelucrează evenimentele: detectare comportament, inițiere acțiuni, notificări.
 - Scalabilitate și timp de răspuns mic.

[SpeedSensor] --->

[Camera] | --> [Event Bus] --> [BehaviorAnalysisService]

[RainSensor] --->

|

v

[ReactionEngine] --> [DashboardService]

Pattern-uri potrivite:

- Builder Pattern – pentru a construi configurații pas cu pas.
- Composite Pattern – pentru a grupa opțiuni în pachete.
- Decorator Pattern – pentru a adăuga funcționalități/configurații extra dinamice.

Avantaje:

- Flexibilitate în adăugarea de noi opțiuni.
- Ușor de extins cu noi pachete fără a modifica codul existent.

Limitări:

- Poate deveni complex la combinarea multor opțiuni.
- Necesită gestionarea consistentă a prețurilor și dependențelor dintre opțiuni.

3. Tabelul employees e central.

- Relații:
 - 1:N între employees și titles, salaries
 - M:N între employees și departments (prin dept_emp)
 - manageri sunt tot employees (prin dept_manager)

Design BLL (Business Logic Layer)

Model orientat pe servicii:

- EmployeeService: CRUD pentru angajați
- SalaryService: gestionare salarii
- DepartmentService: gestionare departamente și manageri
- TitleService: istoric titluri
- AssignmentService: relații între employee și departamente

Avantaje:

- Separare clară a responsabilităților
- Posibilitate de reutilizare și testare unită

Dezavantaje:

- Necesită sincronizare între servicii
 - Tranzacțiile distribuite pot deveni complexe
-

Subiecte 2 5

3

Modelul de domeniu oferit:

- **Ierarhie de moștenire:** Member → Redactor → Admin
- Relații:
 - Comment este scris de un Member
 - Article este scris de un Redactor, aparține unei Section
 - Comment aparține unui Article

Model relațional:

Member (

member_id PK,

name,

email,

member_type ENUM('Member', 'Redactor', 'Admin') -- Single Table Inheritance

)

Article (

article_id PK,

date,

title,

content,

rating,

redactor_id FK → Member.member_id, -- doar dacă member_type = 'Redactor'

section_id FK → Section.section_id
)

Comment (
comment_id PK,
date,
text,
member_id FK → Member.member_id,
article_id FK → Article.article_id
)

Section (
section_id PK,
title
)

- S-a folosit **Single Table Inheritance**: toate tipurile de utilizatori într-un singur tabel (Member), diferențiate printr-un câmp member_type.
- Este eficient și simplu pentru interogări comune și asigură integritatea referințelor.

4.

Designul unui joc cu personaje, arme și apărare

Cerințe:

- Joc cu **personaje variate** (ex: ninja, fantome)
- Fiecare personaj are:
 - **arme multiple** (săbii, cuțite etc.)
 - **mijloace de apărare multiple** (scuturi, vrăji etc.)

- Sistem extensibil pentru a adăuga **noi tipuri de personaje/arme/apărări**

Pattern-uri potrivite:

- **Strategy Pattern**: pentru arme și apărări (comportamente interschimbabile)
- **Composite Pattern** (opțional): pentru combinarea apărărilor/atacurilor
- **Factory Method** sau **Abstract Factory**: pentru creare de personaje cu configurații specifice

Justificare:

- **Strategy Pattern** permite schimbarea dinamică a comportamentului fără modificarea claselor GameCharacter.
 - **Extensibilitate crescută**: adăugarea de noi arme/defensive nu afectează clasele existente.
 - Poți combina aceste cu Abstract Factory pentru a crea „kit-uri” de ninja, ghost etc.
-

Subiect 6 partial discutat in discutie 2

1

Descriere comportament:

- Comanda are stări: Unprocessed, Rejected, Accepted, Pending, Fulfilled.
- Tranzițiile se fac pe baza validității și disponibilității itemilor.

Pattern recomandat:

- **State Design Pattern**

Clasa diagramă (descrișă textual):

```

+-----+           +-----+
| Order   |<----->| OrderState |
+-----+           +-----+
| id       |           | +process() |
+-----+           +-----+
| items    |           +-----+

```

```

| currentState |
+-----+
|
+---> UnprocessedState
+---> RejectedState
+---> AcceptedState
+---> PendingState
+---> FulfilledState

```

Permite separarea logicii pe stări individuale.

Fiecare stare controlează ce se întâmplă în pasul următor.

Ușor de extins cu noi stări.

2 discutat

3

Sistem automat de notare pentru teme

Flux:

1. Studentul trimite o temă.
2. Sistemul verifică plagiatul.
3. Dacă e originală → verifică corectitudinea.
4. Dacă e corectă → notează „pass”, altfel „fail”.
5. Notifică studentul.

Arhitectură recomandată:

- **Layered Architecture + Chain of Responsibility (pentru procesare)**
- **Servicii:**
 - PlagiarismChecker
 - SolutionGrader

- NotificationService

[Student] ---> [SubmissionService]

|

[PlagiarismChecker] --> [Grader] --> [GradebookService] --> [NotificationService]

- Separare clară a responsabilităților.
- Poți adăuga validări noi (ex: stil de cod) ca noi etape în „chain”.
- Poate fi ușor testat și extins.